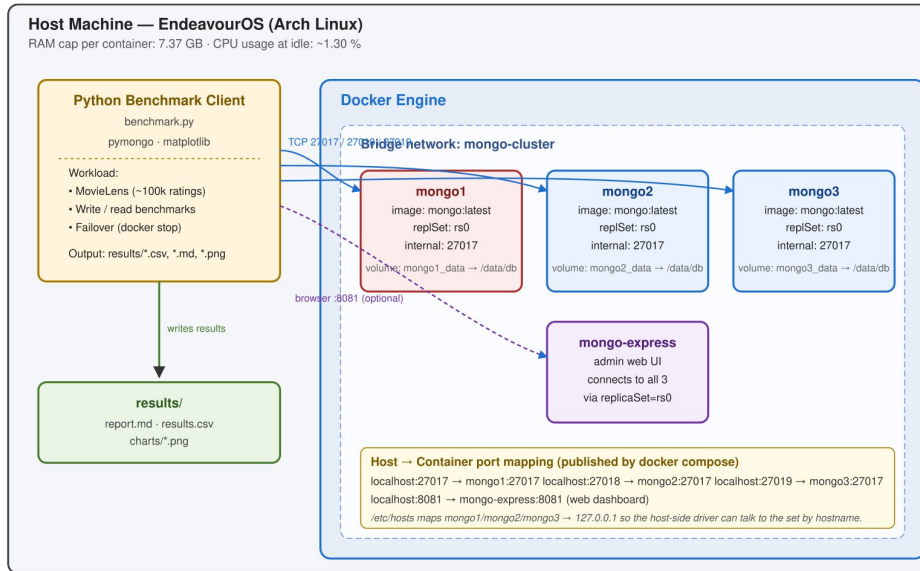


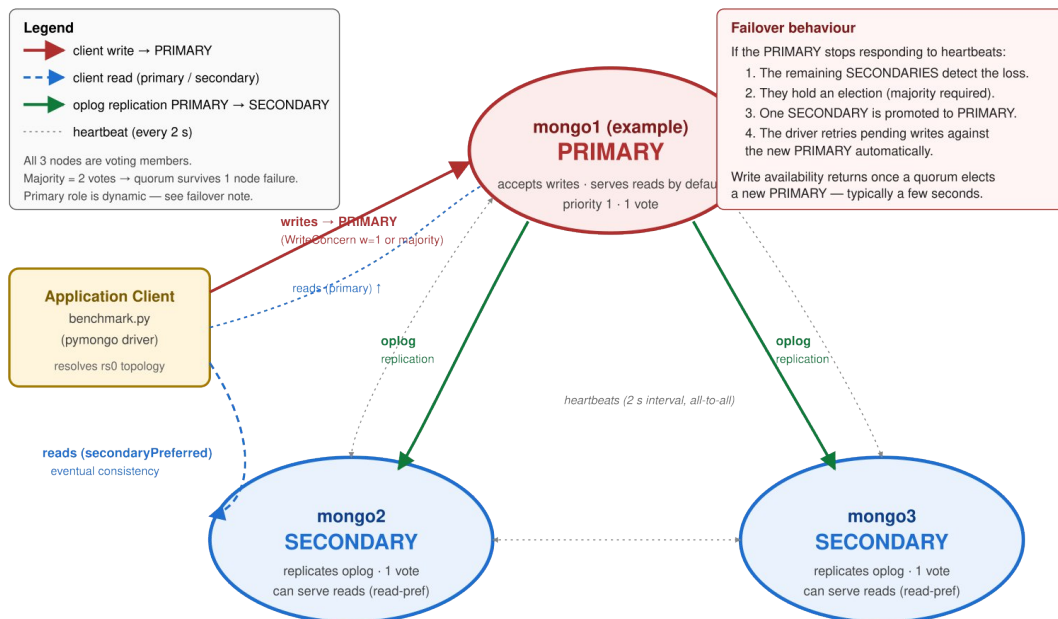
Intermediary work report

1. Architecture Diagram

MongoDB Replica-Set Distributed System — Architecture Overview



Replica Set rs0 — Node Interactions & Network Flows



2. Configuration Details

The framework is configured entirely through the Docker Compose file at “~/Documents/DSE/docker-compose.yaml” and a single one-time “rs.initiate()” call inside “mongosh”. Each “mongod” is started with the flags “--replSet rs0”, “--bind_ip_all” and “--port 27017”. The first argument tells the daemon it is a member of the replica set whose “_id” is “rs0”. “-- bind_ip_all” causes it to listen to on every network interface, which is required so that other containers on the “mongo-cluster” bridge network can reach it. The port assignment is MongoDB’s default port and is the same inside every container

Each Mongo container publishes its internal port “27017” on a distinct host port so that processes outside Docker can address each node individually:

Container	Internal port	Host port
mongo1	27017	27017
mongo2	27017	27018
mongo3	27017	27019
mongo-express	8081	8081

In addition, each container has a named volume bind-mounted to “/data/db” inside the container. This means that the data files survive container restarts and “docker compose down” – only if we add “-v” at the end it would wipe the data. The replica-set configuration (members, terms, election state) is part of that data, so the set does not need to be re-initialized after a normal restart.

For easy visualization of the data, we use “mongo-express”, which is Mongo’s administration UI. It connects to all three nodes through the connection string “mongodb://mongo1:27017,mongo2:27017,mongo3:27017/?replicaSet=rs0”

Below, we can find the configuration of “mongo1” is representative of all three nodes; the others differ only in their service name, container name, hostname, and host port:

{yaml

mongo1:

image: mongo:latest

container_name: mongo1

hostname: mongo1

```

restart: unless-stopped

command: ["mongod", "--replSet", "rs0", "--bind_ip_all", "--port", "27017"]

ports:

  - "27017:27017"

volumes:

  - mongo1_data:/data/db

networks:

  - mongo-cluster
}

```

After the containers come up, the set is formed by a single command issued inside the "mongo1" shell:

```

{js
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "mongo1:27017" },
    { _id: 1, host: "mongo2:27017" },
    { _id: 2, host: "mongo3:27017" }
  ]
})
}

```

After this call, "rs.status()" reports "ok: 1", one member with "stateStr: "PRIMARY"", and the other two with "stateStr: "SECONDARY"", which is the state I observed and screenshotted in Phase 1.

3. Implementation Details

The Phase 3 application is a Python benchmarking client implemented as a single script, "benchmark.py", that exercises the replica set through the official "pymongo" driver. The design is intentionally a thin layer: each benchmark is a tight Python loop that calls one MongoDB operation and records its wall-clock latency, with all timing

collected per-operation so that distribution statistics (not only averages) can be reported.

The script uses the MovieLens latest-small dataset published by GroupLens, which contains roughly 100 000 ratings, 9 700 movies, and 3 600 tags applied by 600 users. It is downloaded and cached on first run. Movies and tags are loaded into the database in full; ~95 % of the ratings are pre-seeded so that read benchmarks have realistic data to query, while the remaining 5 000 ratings are held back as a workload buffer for the write benchmark, so that every measured insert produces a new document instead of overwriting an existing one. Two secondary indexes are created ("movieId: 1}" and "{userId: 1}" on "ratings", plus "{genres: 1}" on "movies") so that the read benchmarks measure index-based performance rather than full collection scans.

The script runs three suites in sequence and records every individual operation. The write suite measures three insertion strategies — single-document inserts ("insertOne" in a loop), bulk inserts in 500-document batches ("insertMany"), and single-document updates ("updateOne"). Each insert pattern is run twice, once with the write concern "w=1" (the PRIMARY acknowledges as soon as the write is in memory) and once with "w='majority'" (the PRIMARY waits until at least two of the three members have acknowledged the write). Comparing the two concerns directly quantifies the latency cost of stronger durability guarantees on this particular cluster.

The read suite measures three query patterns — point lookup by primary key ("find_one({"_id": ...})"), an indexed equality query that returns a bounded result set ("find({"genres": ...}).limit(50)"), and a multi-stage aggregation that computes the top ten highest-rated movies with at least twenty ratings ("group" → "\$match" → "\$sort" → "\$limit"). Each query pattern is run twice, once with the read preference "primary" (reads served by the PRIMARY only — the default, strongly consistent) and once with "secondaryPreferred" (reads served by a SECONDARY when one is available — eventually consistent, but the read load is spread across nodes).

The failover suite measures availability under a controlled failure. The script first identifies which node is currently PRIMARY by asking the driver, then runs 20 warm-up write probes spaced 100 ms apart. It then stops the PRIMARY container with "docker stop" (or, if Docker is not callable from the script's environment, prompts the operator to do so in another terminal). After the stop signal, the script continues to probe with a write every 100 ms and records the latency and the success/failure of each probe. Probes immediately after the kill fail with a connection error because the driver cannot reach the deposed PRIMARY; once the surviving SECONDARIES elect a new PRIMARY, the driver discovers it, the probes start succeeding again, and the script reports the time from the kill to the first successful write as the write downtime. Finally

the script restarts the stopped container and confirms via "rs.status()" that it rejoins as a SECONDARY.

The same set of metrics is collected for every benchmark and reported in a Markdown table at the end of the run:

- **Throughput**, in operations per second, defined as the total number of operations issued during a test divided by the wall-clock time the test took. This measures how many requests the cluster can sustain in this configuration.
- **Latency**, in milliseconds, reported as the mean, the median (p50), the 95th percentile (p95), the 99th percentile (p99) and the maximum. Latency is the wall-clock interval between the driver-level call and its return. Percentiles are reported because the mean alone is misleading for distributed systems: p95 and p99 are what reveal tail behaviour (e.g. a single slow write blocking on a majority-write quorum).
- **Error count**, the number of operations in the test that returned an exception rather than completing — useful as a sanity check that the measured throughput is real and not the result of fast failures.
- **Write downtime for the failover test**, defined as the time elapsed between the moment the PRIMARY's container received the "stop" signal and the moment the first subsequent write probe succeeded against the newly elected PRIMARY.

At the end of a run the script writes three artifacts to "results/":

- "results.csv" — every single operation with its test name, operation index, latency in milliseconds, success flag, and (for failover probes) the phase ("pre_kill" / "post_kill"). This is the raw measurement and can be re-analysed with any spreadsheet or statistics tool.
- "report.md" — a readable summary with one table per benchmark showing the metrics described above, plus a dedicated failover section reporting the write downtime.
- charts/*.png" — four bar/line charts: write throughput by test, read throughput by test, p50 / p95 / p99 latency comparison across all benchmarks, and a failover timeline showing per-probe latency over the full kill-and-recover window with the kill point marked.

Together the suites cover the four scenarios typically expected from a replica-set evaluation: (1) normal throughput under a heavy mixed workload, (2) consistency cost through the "w=1" vs "w='majority'" and "primary" vs "secondaryPreferred" comparisons, (3) load distribution through the secondary-read preference, and (4) fault tolerance through the controlled-kill failover test.

4. Functionality Verification and Testing

The second round verified that the Python benchmark client communicates correctly with the cluster and that all three benchmark suites can be executed end-to-end.

The most notable error was uncovered during the first attempt to run the script against the live stack. The benchmark client failed at the connection step with:

```
{
ServerSelectionTimeoutError: mongo1:27017: [Errno 111] Connection refused
    mongo2:27017: [Errno 111] Connection refused
    mongo3:27017: [Errno 111] Connection refused
FATAL: no node responded as primary. Is the stack up?
}
```

The wording suggested that the cluster was down, but "rs.status()" from inside "mongo1"'s shell continued to report "ok: 1" with all three members healthy, which ruled out a real cluster failure. The diagnostic "docker ps --format "table {{.Names}}\t{{.Ports}}"" revealed the actual problem:

```
{
mongo1      27017/tcp
mongo2      0.0.0.0:27018->27017/tcp
mongo3      0.0.0.0:27019->27017/tcp
}
```

The "mongo1" container was running and was reachable from the other containers over the "mongo-cluster" bridge network, but its port 27017 was not published to the host. Confirming with "ss -tlnp" showed only 27018 and 27019 listening on the host's loopback interface — nothing on 27017 — which is exactly what "Errno 111 Connection refused" from the kernel means: nothing is bound there. The root cause was that the "mongo1" container had been recreated outside Compose at some earlier point (a manual "docker run" or "docker start" against a stripped-down configuration), losing the "ports:" declaration from the Compose file.

The fix was to bring the stack back into a state that matches the Compose file:

```
{bash
```

```
cd ~/Documents
```

```
docker compose down
```

```
docker compose up -d
```

```
}
```

Re-running "docker ps" after the "up" showed the expected mapping on all

three nodes:

```
{
```

```
mongo1  0.0.0.0:27017->27017/tcp
```

```
mongo2  0.0.0.0:27018->27017/tcp
```

```
mongo3  0.0.0.0:27019->27017/tcp
```

```
}
```

After this fix the benchmark client connected successfully. A second, related obstacle was that even with all three host ports published, the replica-set members advertise themselves to the driver by the container hostnames "mongo1"/"mongo2"/"mongo3", which do not resolve on the host's DNS by default. This caused the driver to discover the topology successfully but then fail to issue follow-up requests because it could not resolve the per-member hostnames. The standard remedy was applied: the line "127.0.0.1 mongo1 mongo2 mongo3" was added to "/etc/hosts" so that the host operating system maps the three container hostnames back to the loopback interface. With this in place the driver opens true replica-set connections to all three members and can perform automatic failover on its own.

The cluster has behaved as documented: writes are accepted on the PRIMARY and propagated to SECONDARIES; reads can be served from either role as configured; and the loss of any single node is detected within a few heartbeat intervals and resolved by automatic election. The script's failover suite, once executed against the now-correctly-published cluster, reports the exact write-downtime window in seconds, which will appear in "results/report.md" after the next run.

Below, you can also find some charts that were outputted by the program to show it ran without any problems:

